

 Latest updates: <https://dl.acm.org/doi/10.1145/3729379>

RESEARCH-ARTICLE

AlphaTrans: A Neuro-Symbolic Compositional Approach for Repository-Level Code Translation and Validation

ALI REZA IBRAHIMZADA, University of Illinois Urbana-Champaign, Urbana, IL, United States

KAIYAO KE, University of Illinois Urbana-Champaign, Urbana, IL, United States

MRIGANK PAWAGI, Indian Institute of Science, Bengaluru, KA, India

MUHAMMAD SALMAN ABID, Cornell University, Ithaca, NY, United States

RANGEET PAN, IBM Research, Yorktown Heights, NY, United States

SAURABH SINHA, IBM Research, Yorktown Heights, NY, United States

[View all](#)

Open Access Support provided by:

IBM Research

University of Illinois Urbana-Champaign

Indian Institute of Science

Cornell University



PDF Download
3729379.pdf
03 April 2026
Total Citations: 3
Total Downloads:
1041



Published: 19 June 2025
Accepted: 01 April 2025
Received: 13 September 2024

[Citation in BibTeX format](#)



ALPHATrans: A Neuro-Symbolic Compositional Approach for Repository-Level Code Translation and Validation

ALI REZA IBRAHIMZADA, University of Illinois Urbana-Champaign, USA

KAIYAO KE, University of Illinois Urbana-Champaign, USA

MRIGANK PAWAGI, Indian Institute of Science, India

MUHAMMAD SALMAN ABID, Cornell University, USA

RANGEET PAN, IBM Research, USA

SAURABH SINHA, IBM Research, USA

REYHANEH JABBARVAND, University of Illinois Urbana-Champaign, USA

Code translation transforms programs from one programming language (PL) to another. One prominent use case is application modernization to enhance maintainability and reliability. Several rule-based transpilers have been designed to automate code translation between different pairs of PLs. However, the rules can become obsolete as the PLs evolve and cannot generalize to other PLs. Recent studies have explored the automation of code translation using Large Language Models (LLMs). One key observation is that such techniques may work well for crafted benchmarks but fail to generalize to the scale and complexity of real-world projects with inter- and intra-class dependencies, custom types, PL-specific features, etc. We propose ALPHATrans, a neuro-symbolic approach to automate *repository-level* code translation. ALPHATrans translates both source and test code, and employs multiple levels of validation to ensure the translation *preserves* the functionality of the source program. To break down the problem for LLMs, ALPHATrans leverages program analysis to decompose the program into fragments and translates them in the *reverse call order*.

We leveraged ALPHATrans to translate *ten* real-world open-source projects consisting of (836, 8575, 2719) (application and test) classes, (application and test) methods, and unit tests. ALPHATrans breaks down these projects into 17874 fragments and translates the entire repository. 96.40% of the translated fragments are syntactically correct, and ALPHATrans validates the translations' runtime behavior and functional correctness for 27.03% and 25.14% of the application method fragments. On average, integrated translation and validation takes 34 hours (min=3, max=121) to translate a project, showing its scalability in practice. For the syntactically or semantically incorrect translations, ALPHATrans generates a report including existing translation, stack trace, test errors, or assertion failures. We provided these artifacts to two developers to fix the translation bugs in four projects. They fixed the issues in 20.1 hours on average (5.5 hours for the smallest and 34 hours for the largest project) and achieved all passing tests. Without ALPHATrans, translating and validating such big projects could take weeks, if not months.

CCS Concepts: • **Software and its engineering** → **Source code generation**; • **Computing methodologies** → **Machine translation**.

Additional Key Words and Phrases: Neuro-Symbolic Code Translation and Validation

Authors' Contact Information: [Ali Reza Ibrahimzada](#), University of Illinois Urbana-Champaign, Urbana, USA, alirezai@illinois.edu; [Kaiyao Ke](#), University of Illinois Urbana-Champaign, Urbana, USA, kaiyaok2@illinois.edu; [Mrigank Pawagi](#), Indian Institute of Science, Bengaluru, India, mrigankp@iisc.ac.in; [Muhammad Salman Abid](#), Cornell University, Ithaca, USA, ma2422@cornell.edu; [Rangeet Pan](#), IBM Research, Yorktown Heights, USA, Rangeet.Pan@ibm.com; [Saurabh Sinha](#), IBM Research, Yorktown Heights, USA, sinhas@us.ibm.com; [Reyhaneh Jabbarvand](#), University of Illinois Urbana-Champaign, Urbana, USA, reyhaneh@illinois.edu.



This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2994-970X/2025/7-ARTFSE109

<https://doi.org/10.1145/3729379>

ACM Reference Format:

Ali Reza Ibrahimzada, Kaiyao Ke, Mrigank Pawagi, Muhammad Salman Abid, Rangeet Pan, Saurabh Sinha, and Reyhaneh Jabbarvand. 2025. ALPHATrans: A Neuro-Symbolic Compositional Approach for Repository-Level Code Translation and Validation. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE109 (July 2025), 23 pages. <https://doi.org/10.1145/3729379>

1 Introduction

Application modernization offers numerous benefits to developers, including better performance, maintainability, productivity, reliability, and security [28, 29, 31, 32]. Manual migration or modernization of real-world projects can be time-consuming and error-prone. Code translation can help automatically convert programs from one programming language (PL) to another.

Transpilers solely rely on program analysis and perform rule-based translation, failing to translate code between languages that greatly differ in syntax or semantics [5]. This also makes them very PL-specific; they cannot generalize to newer features of the same PL pairs easily, let alone other PLs. Finally, the translations lack readability, requiring much effort to understand and validate them, and naturalness, failing to create idiomatic code in the target PL [44]. State-of-the-art code translation techniques attempt to harvest the emerging abilities of Large Language Models (LLMs) in code synthesis to overcome the limitations of transpilers [42, 44, 62]. However, these techniques are still limited to translating simple programs in crafted benchmarks or selected slices of real-world projects due to the following challenges:

- (1) *Problem complexity*. The source and target PLs can be fundamentally different in programming paradigms, typing, and memory management. Some PLs have specific properties that may not exist in others, e.g., constructor overloading in Java. Such complexities are beyond the abilities of existing LLMs to handle, causing them to hallucinate when translating types, code constructs, or even method names [44], making translations non-compilable or useless.
- (2) *Validation*. The translation should preserve the functionality of the source project. Most existing techniques follow a “translation first and validation next” approach, which can postpone the validation and not benefit from the potential use of validation as feedback to correct the translation [44]. A few techniques use formal methods [42, 62] to verify translations on the go. However, these techniques cannot scale to real-world projects. One possible solution for validation is reusing the tests in the source language. However, due to (1) multiple invocations of different methods in unit tests and (2) inherent long call chains in real-world projects, testing a translated method *in isolation* is impossible.
- (3) *Limited context window*. Concerning repository-level translation, the entire project and, in many cases, even the entire class cannot fit into the context window of current LLMs [25]. Assuming an unlimited context window, LLMs still suffer from short attention span [38], preventing them from properly capturing the intra- and inter-procedural dependencies in real-world projects.

This paper presents ALPHATrans, a neuro-symbolic¹ approach for automated repository-level code translation and validation. ALPHATrans leverages static analysis to resolve PL-specific features of the source language (§4.1), decompose the source project into smaller fragments (§4.2), and create a compilable project skeleton in the target language (§5). It then starts translating fragments in reverse call order and validates them using existing tests when possible (§6). After translating each fragment, ALPHATrans updates the project skeleton and ensures the whole project compiles, gradually translating and validating the source project into the target PL. To improve translation quality, static analysis again comes to the rescue: ALPHATrans collects relevant context for each

¹The keyword symbolic here refers to a general term of symbolic learning in contrast to machine learning and should not be confused with symbolic execution. We refer to combining LLMs and program analysis as a neuro-symbolic approach.

fragment, including translated callee methods and surrounding contexts, e.g., class declaration, global variables/fields, etc. It also uses relevant in-context examples based on the specific properties of the fragment to be translated. ALPHATrans implements two types of dynamic validation: (1) running the source tests on the translated fragments in isolation using language interoperability (§6.1) and (2) decomposing, translating, and executing the source tests on the translated fragments (§6.3). Finally, ALPHATrans recomposes the translated fragments to create the program in the target PL (§6.2).

Our approach of compositional translation and validation is PL-agnostic; however, implementing the program transformation component is PL-specific. For the first version of ALPHATrans, the implementation supports translating Java code to Python. Our motivations for choosing this PL pair are: (1) Java offers many features that are not supported or common in other PLs by default (e.g., method/constructor overloading, complex types, circular dependencies, local or anonymous inner classes, interfaces, etc.); (2) Python programs are not compiled but interpreted, which makes many translation issues that can be caught at the compile time remain undetected until test execution and challenge the validation; and (3) both PLs are popular (top-5 on the TIOBE index [56]).

Using ALPHATrans to translate ten real-world Java projects to Python corroborates its *effectiveness*: It can translate 17874 field/method/test fragments, with 96.40% syntactic correctness. For the 4654 application method fragments that can be further evaluated through test execution, ALPHATrans achieves 27.03% runtime validation and 25.14% functional equivalence using the source tests. ALPHATrans is *scalable*, completing translations in 34 hours, on average. Human subjects improved partial translation of ALPHATrans and achieved passing test suites within 20.1 hours, on average, showing *practicality* of ALPHATrans. These results were achieved using a moderate-size open-access LLM (DeepSeek-Coder-33b-Instruct [24]). A stronger model, i.e., GPT-4o, improves the performance of ALPHATrans to 99.2% syntactic correctness for all fragments and 27.95% functional equivalence for method fragments, with an overhead of \$14.39 per project, on average. The affordable cost is due to the novel features of the pipeline, namely, decomposition into fragments, prompt crafting, in-isolation validation of translations, and efficient feedback loop.

To the best of our knowledge, ALPHATrans is the *first technique to translate an entire repository*, including tests, and generates validated translations (considering existing tests). The only prior repository-level translation attempt using GPT-4 [44] (translating Apache Commons CLI from Java to Python) resulted in non-compilable code, let alone the translation being validated. ALPHATrans is also *the first technique leveraging language interoperability for in-isolation validation of translated fragments*. The effort of human subjects to fix translation bugs by ALPHATrans and achieve passing tests creates pragmatic bug data sets for testing, bug localization, and program repair research. Our code and artifacts are publicly available for reproducing the results or translating new projects [33].

2 Challenges in Repository-Level Code Translation

To illustrate the most notable challenges in repository-level code translation and validation, we use the hypothetical example in Figure 1, inspired by the complexities in real-world Java projects.

Challenge 1: Class Size. The class consists of 25,380 tokens (a). Instructions for translating the code, in-context examples, and the model’s response can also significantly increase the number of input tokens. While some commercial LLMs support tens of thousands of tokens, many open-access LLMs do not. For example, DeepSeek-Coder-33b-Instruct [24] used in this paper has a context window of 16,384 tokens, of which only 4,096 tokens can be used for generation. To address this challenge, ALPHATrans decomposes Java application classes into smaller *field* and *method fragments* and translates each separately in reverse call order (§4.2.1, §4.2.2).

Challenge 2: PL-specific Properties. Java programs frequently use method/constructor overloading, which is not supported directly in Python (a). This example shows instances of constructor

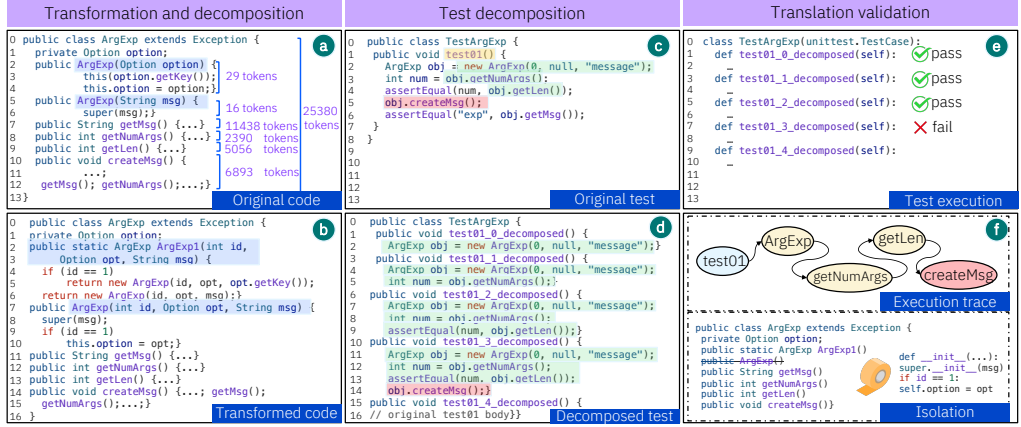


Fig. 1. Illustration of key challenges in repository-level code translation and ALPHATrans addressing them.

overloading (lines 2 and 5). In Python, declaring two constructors is allowed; however, at runtime, the last declaration overrides all previous constructors, resulting in unexpected behavior. To address this issue, ALPHATrans employs program analysis to refactor the original code while preserving the functionality (through test execution). The transformation includes changing the constructor's name, updating the references, and changing the constructor's implementation. The transformed code (b) makes the source program amenable to translation to Python.

Challenge 3: Validation. To illustrate the challenges with validation, consider test01 (c) that invokes *four* methods in its body (ArgExp, getNumArgs, getLen, and createMsg) to test the functionality of method getMsg in the assert statement. Suppose we can successfully translate all methods except createMsg. If we choose test translation (a natural way of validating code translation), the execution of the translated test results in a runtime error when invoking createMsg. As a result, a translation issue in one method casts a shadow in validating the translation of the other methods. We refer to this issue as the *test translation coupling effect*. To overcome this challenge, ALPHATrans executes source language tests as-is (i.e., without translation) by leveraging a language-interoperability framework called GraalVM [43] (f). In this setting, a test in the source language is executed each time one of its invoked application methods (method fragments) is translated. This approach validates *functional equivalence* of each method *in isolation* as the other invoked application methods during test execution remain in the source language.

Challenge 4: Test Translation. GraalVM has certain limitations (§6.1), which prevents ALPHATrans from validating all the code fragments in isolation. Furthermore, we need to translate tests regardless of whether they are used for validation to maintain the translated projects in the target language. Test errors due to test translation coupling effect under-approximate the quality of translation: failing to validate the translation of four methods because of one incorrect translation. To overcome this challenge, ALPHATrans decomposes the original test suite into *test fragments* (d). Executing the translated decomposed test suite results in three test passes (e), validating the *runtime behavior* of three methods that the original test suite could not promptly provide.

An alternative approach is parsing the stack trace and code coverage results for each runtime error during translation. However, test decomposition is a cleaner way to see the results per test execution promptly. It is also done once before translation. In translation to interpreted languages such as Python, specifically, the execution of test fragments can validate the runtime behavior of methods before waiting for functional validation. For fragments that GraalVM cannot validate, if ALPHATrans can successfully translate all the methods invoked during test execution and the test passes, such a test will also be used for validating *functional correctness*.

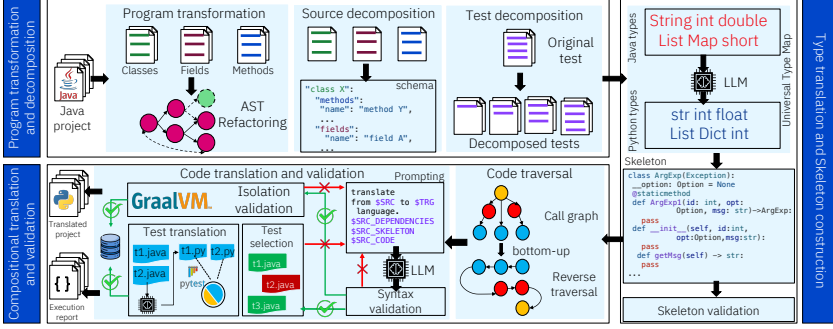


Fig. 2. Overview of ALPHATrans.

3 Overview of Approach

ALPHATrans consists of three main phases, shown in Figure 2: program transformation and decomposition (§4), type translation and skeleton construction (§5), and compositional translation and validation (§6). The first two phases aim to decompose and simplify the repository-level code translation problem for LLMs, helping the third phase yield high-quality validated translations.

The program transformation and decomposition phase first refactors the PL-specific properties of the source program into programming paradigms common among many PLs (§4.1). Next, it decomposes the source project into smaller units, i.e., *fragments*, and stores fragment dependencies in a data structure called *schema* (§4.2).

The type translation and skeleton construction phase takes the schema as input and produces *target project skeleton*, i.e., a compilable project in the target language with method signatures but no method implementation (§5.2). The first translation step happens here, where the source PL types are translated to the target PL to ensure that class skeletons are compilable (§5.1). The outcome of type translation is a type mapping from the source to the target PL, which ALPHATrans can reuse in translating other projects.

The compositional translation and validation phase takes the schema and project skeleton as inputs and translates fragments, in reverse call order, by prompting an LLM. After translating a fragment, it updates the class skeleton with a new translation and checks whether the skeleton compiles. For a method fragment, ALPHATrans looks for corresponding tests and, if any exist, uses them to validate the fragment. The first level of validation is performed through GraalVM’s language interoperability to isolate the validation of the method using tests in the source language. Next, ALPHATrans translates and executes the tests. In case of compilation errors or test failures, ALPHATrans reprompts the LLM with feedback (from the compilation/runtime errors) to improve the translation. If no improvement is achieved within a certain budget, ALPHATrans continues to the next fragment until all are translated. For methods whose translations are not compilable or result in test errors/failures, ALPHATrans generates reports consisting of existing translations and relevant artifacts, such as stack traces, test errors/failures, and test coverage information.

4 Program Transformation and Decomposition

4.1 Program Transformation

This component performs semantics-preserving refactoring of method and constructor overloading in Java code to make it amenable to translation to Python. Other Java-specific features, namely, circular dependencies, inner classes, interfaces, and abstract classes, are handled while constructing the project skeleton in Python (§5.2). The reason for resolving method and constructor overloading in the source language is that we have to change the implementation, i.e., call sites to methods and constructors. Therefore, such changes should be validated using source tests before translation.

<pre> 1 public ArgExp(Option opt) { 2 this.option = opt; 3 this.threshold = 1; 4 } 5 public ArgExp(String msg) { 6 this.msg = msg; 7 } </pre> Original code	<pre> 1 public ArgExp(Option opt) { 2 this(opt.getKey()); 3 } 4 public ArgExp(String msg) { 5 super(msg); 6 } </pre> Original code	<pre> 1 public ArgExp(Option opt) { 2 this(opt.getKey()); 3 this.option = opt; 4 } 5 public ArgExp(String msg) { 6 super(msg); 7 } </pre> Original code
<pre> 1 public ArgExp(int id, Option opt, String msg){ 2 if (id == 0) 3 { 4 this.option = opt; 5 this.threshold = 1; 6 } 7 else { 8 this.msg = msg 9 } </pre> Transformed code	<pre> 1 public static ArgExp ArgExp1(Option opt) { 2 return new ArgExp(opt.getKey()); 3 } 4 5 public ArgExp(String msg) { 6 super(msg); 7 } </pre> Transformed code	<pre> 1 public static ArgExp ArgExp1(2 int id, Option opt, String msg){ 3 if (id == 1) 4 { 5 return new ArgExp(id, opt, opt.getKey()); 6 return new ArgExp(id, opt, msg); 7 } 8 public ArgExp(int id, Option opt, String msg){ 9 super(msg); 10 if (id == 1) 11 this.option = opt; </pre> Transformed code
(a)	(b)	(c)

Fig. 3. Constructor overloading patterns and their corresponding transformations.

For overloaded methods, ALPHATRANS makes each method name unique by adding an integer suffix (starting at 0) to the name, and updates all call sites based on the new method names. Resolving overloaded constructors is not as straightforward, as they should have the same name as the enclosed declaring class. Furthermore, the invocation of constructors inside each other and the Java inheritance mechanism makes constructor overloading complex. Our algorithm (Algorithm 1) for resolving the constructor overloading handles three prominent patterns shown in Figure 3.²

The first pattern (Figure 3-a) shows multiple independent constructors. ALPHATRANS merges these constructors into one and uses an id parameter to differentiate between them. All call sites of the constructors are updated accordingly to use the appropriate id value. The second pattern (Figure 3-b) involves a constructor call chain using this(). ALPHATRANS transforms the first constructor into a factory method and invokes the second constructor inside it. Factory methods are static, and ALPHATRANS updates the call sites to invoke them directly on the class, e.g., ArgExp.ArgExp1(id,opt,msg). The last pattern (Figure 3-c) is similar to the second one, except that both constructors implement some code. ALPHATRANS refactors the first constructor into a factory method and adds an extra id parameter to differentiate between behaviors implemented by different constructors. The constructor call sites are updated accordingly, as in the previous cases. Real-world projects often combine these patterns, which ALPHATRANS handles using Algorithm 1.

Algorithm 1: Constructor Overloading

Inputs: Overloaded Constructors *OCs*
Output: Code without Overloaded Constructors *NOCs*

```

1 if !hasThisCall(OCs) then
2   ids ← createConstructorIDS(OCs);
3   NOCs ← merge(OCs, ids);
4 else
5   if hasOnlyThisCall(OCs) then
6     refactor ← createRefactorMethod(OCs);
7     NOCs ← merge(OCs, refactor);
8   else
9     refactor ← createRefactorMethod(OCs);
10    ids ← createConstructorIDS(OCs);
11    NOCs ← merge(OCs, refactor, ids);
12 refactorCallSites();
13 return NOCs;

```

4.2 Program Decomposition

Translating the entire repository of real-world projects is a very complex problem. As a result, ALPHATRANS breaks down projects into fragments, performs the translation and validation at the fragment level, and re-composes the translation as a repository in the target language.

4.2.1 Source Decomposition. Real-world projects can include hundreds of files with thousands of lines of code, which exceed the context window of state-of-the-art LLMs. ALPHATRANS employs static analysis to decompose code into smaller fragments, i.e., *field fragments* and *method fragments*. A field fragment includes modifiers, type, name, and field value. A field fragment can belong to an application or test class. A method fragment includes the method signature and can be an

²Following the best practices for constructor overloading from *Stack Overflow* and analyzing the use of constructor overloading in open-source projects, we categorized the use cases into the three patterns.

application or test method (e.g., helper methods or unit tests). During decomposition, ALPHATrans extracts meta-information related to the fragments, such as their location (e.g., start and end line), code (e.g., implementation between start and end line), dependencies (e.g., callers and callees), types (of inputs and output), and other necessary information such as file paths, class inheritance, imports, and method annotations. ALPHATrans stores fragments and their corresponding collected meta-data in a data structure called *schema*, which is used in the other phases. ALPHATrans also extracts the call graph to guide the translation, i.e., to translate fragments in reverse call order.

4.2.2 Test Decomposition. The burden of checking *functional equivalence* in ALPHATrans is on GraalVM. Yet, we still need to translate and execute tests to validate the fragments that cannot be validated by GraalVM (§6.1). Unit tests in real-world projects can invoke multiple methods and include multiple assert statements. Furthermore, long call chains are inevitable in real-world projects due to the high degree of intra- and inter-procedural dependencies. As we show later (§7.4), the average number of direct method invocations and method executions in tests for our studies subjects are 3 and 27, respectively. This can result in *test translation coupling effect*, discussed in §2.

To enable test translation for runtime validation or checking functional equivalence, ALPHATrans decomposes each unit test into a series of *test fragments*, as shown in Figure 1-d. It uses each statement with a call to an application method as a cutting point. For statements enclosed by branches, loops, or exception-handling blocks, ALPHATrans includes the entire block. This process generates an ordering of executable test fragments for each unit test. Each test fragment includes all the statements of the lower-order fragments, along with additional statements that invoke one additional method not invoked by previous fragments. ALPHATrans executes test fragments in increasing order until a test fails and skips running following fragments, as they will also fail.

5 Type Translation and Skeleton Construction

5.1 Type Translation

Automatically resolving types is a challenging problem [23, 55], and a large body of work has attempted to address this, mostly using symbolic rule-based approaches [7, 10, 21, 35, 47, 48]. ALPHATrans employs a Retrieval-Augment Generation (RAG) [37] technique for finding equivalent types in the target language. To that end, it first extracts all the types in the source language of a given project. Custom application types are resolved during the translation as ALPHATrans translates fragments and classes in the target language. For the remaining types, it crawls the API documentation of the source language and retrieves the relevant description of each type.

To form the prompt,³ ALPHATrans uses the retrieved description and instructs the model with an in-context example to return the most relevant type in the target language, given the use of types in the source language and the retrieved description. To account for potential hallucination in LLM's response, i.e., returning a type that does not exist in Python, ALPHATrans employs a simple Python script, uses the translated type as an annotation, executes the script, and keeps the ones that have no syntactic or runtime issue. The types in the source language and their corresponding in the target language form a data structure called *universal type mapping*. In practice, ALPHATrans reuses or augments the mapping when translating new projects.

```

0 class ArgExp(Exception):
1     __option: Option = None
2     @staticmethod
3     def ArgExp1(id: int, opt: Option, \
4                 msg: str) -> ArgExp:
5         pass
6     def __init__(self, id: int, opt: \
7                 Option, msg: str):
8         pass
9     def getMsg(self) -> str:
10        pass
11    def getNumArgs(self) -> int:
12        pass
13    def getLen(self) -> int:
14        pass
15    def createMsg(self) -> None:
16        pass

```

Fig. 4. Target PL skeleton for the example in Figure 1-b.

³Due to space limit, we omit the prompt; please refer to our artifacts [33] to see the prompts used for type resolution.

5.2 Skeleton Construction

ALPHATrans builds the project's structure in the target language before translation. This step is necessary for compositional translation and validation, as ALPHATrans can insert the translated fragments into the project, compile it, or even execute the existing translated test suites, gradually completing the translation. At this step, ALPHATrans also resolves Java-specific features in Python before starting the translation. Specifically, it resolves circular imports and dependencies, inner classes, interfaces, and abstract classes. Figure 4 shows the class skeleton corresponding to the illustrative example of Figure 1-b.

At the first step, ALPHATrans creates Python classes corresponding to each application class in Java. The fields in the classes are set to None, and ALPHATrans uses the information in schema (§4.2.1) to ensure the naming corresponds to the type of their access modifier in the source language. In the example of Figure 4, the translation of field `private Option option;` in Java is `__option: Option = None`. The classes also include method signatures, with their body set to pass. ALPHATrans uses the universal type mapping (§5.1) to create relevant types in the method signature. Once the initial skeleton is created, ALPHATrans leverages the extracted call graph during program decomposition to detect circular dependencies. If such dependencies exist, ALPHATrans resolves them with local imports. For inner classes, ALPHATrans unfolds them in Python and uses *dot notation* to access specific methods and fields (e.g., `Class.methodName`). Finally, ALPHATrans implements best practices in Python and subclasses all abstract classes and interfaces from `abc.ABC` class. Here, `ABC` is a class from the `abc` module in the Python standard library, which is used for defining abstract base classes.

6 Compositional Translation and Validation

ALPHATrans translates method fragments in reverse call order. It takes the project's call graph, removes the back edges to make it acyclic, computes topological order (i.e., linear ordering of its vertices), and translates method fragments (corresponding to vertices) in reverse topological order. Field fragments are independent; therefore, AlphaTrans translates them before method fragments. The algorithm takes a fragment F , LLM M , Skeleton S , and a series of parameters as inputs, translates the fragment, recomposes the skeleton with the successfully translated fragment, and returns translation outcomes: (1) syntax check (denoted by the “non-parseable” and “parseable” labels), (2) functional equivalence check (denoted by the “graal-fail”, “graal-success”, and “graal-error” labels), and (3) translated test execution check (denoted by “not-exercised”, “test-fail”, and “test-success” labels).

ALPHATrans employs iterative and feedback-based prompting. That is, if one of the mentioned checks fails, e.g., the translated fragment is not syntactically correct, it prompts the model for another translation. To control the number of iterations, ALPHATrans uses a reprompting budget (*repromptBudget*). The algorithm takes the minimum (*min_{budget}*) and maximum (*max_{budget}*) values for the budget and dynamically sets reprompting budget to a number between the lower and upper bounds based on coverage information, e.g., the budget is close to *max_{budget}* for a fragment if it is exercised multiple times (high hit rate based on coverage information) by different unit tests. The rationale is to give more importance to fragments covered by more tests to eventually increase translation validation success.

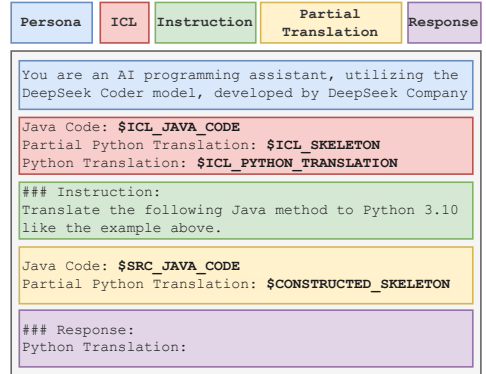


Fig. 5. Prompt structure in ALPHATrans.

Algorithm 2 shows the main translation and validation loop (lines 3–31), which runs until the reprompting budget is exhausted. Inside the loop, ALPHATrans first crafts a unique prompt based on the template shown in Figure 5 and then instructs the LLM to translate the fragment (lines 4–5). It then validates the generated translation in multiple steps. The first step checks for syntactic correctness and assigns proper labels to *TVO* (lines 6–10). Then, ALPHATrans leverages GraalVM for isolation-based validation of fragment *F* (lines 12–20), if there exists a test in the source language covering the fragment during its execution.

Finally, it translates and executes decomposed fragment tests: if there are no eligible tests (a test becomes eligible if all its dependencies are translated) for the fragment, ALPHATrans simply assigns the “*not-exercised*” label to the fragment and moves on to the next one (lines 21–23). Otherwise, it translates the tests, executes them to validate the fragment, and assigns test outcome labels in *TVO* (lines 24–31). In case of a test failure, ALPHATrans extracts all involved fragments and reprompts them with feedback extracted from test execution.

Due to inherent intra- and inter-procedural dependencies in real-world projects, the number of fragments involved in reprompting could be high, logarithmically increasing the translation time. ALPHATrans filters out those with GraalVM label “*graal-success*”, ranks the remaining based on suspiciousness score, and reprompts *topK* suspicious fragments. The suspiciousness score for fragments is calculated such that a fragment with more failing tests will get a higher score and, therefore, ranked higher among other fragments.

Our prompt template consists of *five* distinct parts (Figure 5). The first part is the *persona* message used by DeepSeek-Coder-33b-Instruct during instruction fine-tuning and is required for producing the best outputs. The next part introduces the *In-Context Learning (ICL)* example, which reflects the complexities of code translation and instructs LLM on how to deal with them. The green part shows the natural language instruction given to the model. After describing the objective, the prompt embeds the source Java code along with *partial translation* as a skeleton, which includes all dependencies and translations of the fragments invoked by the current one. The prompt concludes with *### Response:* keyword to guide the model for code generation.

6.1 Language Interoperability

GraalVM [43, 60] is a Java Development Kit by Oracle. It offers the *Polyglot* API [22], which allows the integration of programs written in different guest languages within a Java-based host application. In the context of this work, GraalVM allows execution of Python code from Java and

Algorithm 2: Main Translation Loop

Inputs: Skeleton *S*, Fragment *F*, Model *M*, *minbudget*, *maxbudget*, and *topk* suspicious methods
Output: Translation and Validation Outcome *TVO*

```

1 feedback ← ∅; TVO ← {};
2 repromptBudget ←
   getAdaptiveBudget(F, minbudget, maxbudget);
3 while repromptBudget > 0 do
4   prompt ← generatePrompt(F, feedback);
5   translation ← translateFragment(prompt, M);
6   if !syntacticCheck(translation) then
7     TVO["syntax_outcome"] ← "non - parseable";
8     feedback ← getFeedback(translation);
9     repromptBudget ← repromptBudget - 1;
10    continue;
11  TVO["syntax_outcome"] ← "parseable";
12  if !graalCheck(translation) then
13    TVO["graal_outcome"] ← "graal - fail";
14    feedback ← getFeedback(translation);
15    repromptBudget ← repromptBudget - 1;
16    continue;
17  else if graalLimitation(translation) then
18    TVO["graal_outcome"] ← "graal - error";
19  else
20    TVO["graal_outcome"] ← "graal - success";
21  if !hasEligibleTests(F) then
22    TVO["test_outcome"] ← "not - exercised";
23    break;
24  testTranslation ← getTestTranslation(F);
25  S ← S ∪ translation;
26  if !testCheck(S, testTranslation) then
27    TVO["test_outcome"] ← "test - fail";
28    repromptSuspiciousMethods(testTranslation, topK);
29    repromptBudget ← repromptBudget - 1;
30    continue;
31  TVO["test_outcome"] ← "test - success";
32  break;
33 return TVO;

```

vice versa [3]. ALPHATrans leverages the Polyglot API to perform *in-isolation* validation of the fragments by replacing the Java implementation of a method with its translated Python version while keeping the rest of the project in Java. It then executes the Java tests covering the fragment to validate the functional equivalence of the translation.

The Polyglot API allows access to Python data objects from Java and vice-versa, as these objects reside in a shared memory space. However, objects must be cast to appropriate types for passing parameters to, and processing returned values from, polyglot calls. The Polyglot API can perform this casting implicitly for only a few simple data types. ALPHATrans builds on top of the Polyglot API to provide a framework to create a Python program state that is isomorphic to the Java program state. The Python translation is restricted to this isomorphic state, and the states are synchronized after method calls to preserve the isomorphism. ALPHATrans allows for the casting of user-defined types as well as several built-in and library types. Using both static and dynamic type information of Java objects, ALPHATrans can disambiguate target types when casting Python objects to Java types. It further preserves object identities and aliasing during such casting and propagates exceptions across language boundaries.

To validate the translation of a method m in isolation, ALPHATrans creates an instrumented version of the Java source code. We refer to this instrumented Java project as the *primal project*, P_m . During instrumentation, ALPHATrans replaces the original Java implementation, m_J of m with a polyglot call to its Python implementation, m_P . m_P resides inside a Python project, which we refer to as the *dual project*, D_m . The structure of D_m is similar to that of the original Java project. All other methods in D_m wrap a call to the corresponding methods in P_m . Doing so provides an interface for m_P to execute with access to all other methods and fields, although these are defined only in P_m . Using the call graph for the Java project, ALPHATrans determines all test methods that invoke m and executes them in P_m to validate the translation m_P . This *in-isolation* validation approach is limited in the sense that it can handle only a limited number of built-in and library types. In certain cases, such as reference cycles involving maps or objects with impure methods for hashing, the isomorphism between Java and Python states may not be maintained. Furthermore, it may sometimes not be possible to disambiguate target types when casting Python objects to Java types, for example, if the target object has type `List<Object>`.

6.2 Target Program Recomposition

ALPHATrans recomposes the project skeleton with syntactically correct translated fragments at each iteration of Algorithm 1, gradually constructing the project in the target PL. The recomposed target program in Python is executed against eligible translated tests, i.e., those that cover only the translated fragments.

6.3 Test Translation

Similar to translating application fragments, ALPHATrans also translates test fragments. Using the dependency information captured during static analysis, it crafts prompts for unit tests along with their dependencies for the model to translate. The ICL examples used for prompting test fragments differ from prompts used for translating application method fragments. The focus of ICL examples here is to prevent the LLM from hallucinating the used assert statements in the source PL to the target PL. To construct ICL examples for test fragment translation, we created a pool of in-context examples, where each example shows the Python assert statements equivalent to Java assert statements in the context of a test. When prompting a test fragment, ALPHATrans detects the assert statement in the fragment and retrieves the corresponding examples from the pool. For translated tests, only syntactic validation is performed as there is no other means of validating their translations.

7 Empirical Evaluation

To evaluate different aspects of ALPHATrans, we investigate the following research questions:

- RQ1:** *Effectiveness of ALPHATrans.* To what extent ALPHATrans can automatically resolve types from source to target PL? Can ALPHATrans effectively translate real-world projects?
- RQ2:** *Translation Bugs and Fixes.* How much effort do developers spend completing the partial translations by ALPHATrans? What is the nature of translation bugs?
- RQ3:** *Impact of Test Decomposition.* What is the impact of test decomposition on validation results?
- RQ4:** *Impact of Test Coverage.* To what extent does a test suite with higher coverage impact the validation results?
- RQ5:** *Ablation Study.* To what extent do program transformation, choice of LLM, and program decomposition impact the performance of ALPHATrans?

7.1 Experiment Setup

We followed three steps for selecting subjects: **1- Mining:** We mined GitHub and retrieved a list of repositories that use Java as the primary language, are self-contained (include build files, etc.), and have more than 30 stars with at least one commit pushed within the last 12 months. **2- Filtering:** We filtered out projects based on the number of call edges in their call graphs: removed those with less than 2,000 call edges to ensure the subject projects are big enough to challenge ALPHATrans. We also removed those with more than 30,000 call edges to reduce the computation and manual effort for further steps. Per GraalVM requirements, we only selected projects we could successfully build (compile and achieve green tests) using Java at 21. **3- Reduction:** ALPHATrans currently supports the following Java APIs: core Java API (`java.util`, `java.text`, `java.lang`, `java.io`, `java.nio`, `java.net`, `java.time`, and `java.math`) and third-party libraries (`org.opentest4j`, `org.slf4j.Logger`, and `org.junit`). We automatically removed all other third-party library dependencies and their usage in the source code in the selected projects. We chose a project if at least 50% of its total methods were preserved after such process. Table 1 shows the list of ten projects used in the evaluation of ALPHATrans and details about their size (classes, methods, tests, and fragments).

ALPHATrans uses CodeQL [20] and tree-sitter [59] for static analysis. For running tests, validating translation, and computing coverage, ALPHATrans uses GraalVM 21.0.3 + 7.1 [43], JUnit 4 and 5 [54], Pytest 8.2.1 [46], JaCoCo [53], and Python’s coverage [6]. ALPHATrans works with API- and open-access LLMs. We considered the following criteria for selecting the LLM: (1) for better reproducibility, we prioritized open-access models; (2) due to computing constraints, we wanted an LLM with moderate size ($> 10B$ but $< 70B$ parameters); (3) the model should perform reasonably well in code-related tasks; and (4) the model should have fast inference time due to the huge number of prompts. Per the mentioned criteria, we selected DeepSeek-Coder-33b-Instruct [24] for the main experiments (RQ1–RQ4). We also used GPT-4o, one of the best-performing commercial models, for RQ5 (§7.6) to demonstrate the impact of stronger models on improving the performance of ALPHATrans. We prompted models with the temperature set to 0 for reproducibility and used their default settings for other parameters. For the base prompting (Algorithm 2), we set the minimum and maximum values of the reprompting budget to 3 and 5. For the feedback prompting, we set the reprompting budget to 1, i.e., ALPHATrans attempts to fix issues with feedback only once.

7.2 RQ1: Effectiveness of ALPHATrans

In this RQ, we evaluate ALPHATrans in (1) type translation and skeleton construction (§7.2.1) and (2) compositional translation and validation (§7.2.2).

7.2.1 Effectiveness in Type Resolution and Skeleton Validation: ALPHATrans extracted 1,797 distinct types from the source projects and attempted to translate them to equivalent Python types. Of

Table 1. Effectiveness of ALPHATrans in program transformation, automated type translation, and skeleton validation. **ATR**: Automated Types Resolution, **SV**: Skeleton Validation. The number of classes and methods include both application and test classes/methods.

Subjects	# Classes	# Methods	# JUnit Tests	Method Coverage (%)	ATR (%)	SV (%)	# Fragments			
							Fields		Application	Test
							Application	Test	Methods	Methods
cli [11]	58	664	437	94.14	96.60	100	104	57	273	2180
codec [12]	156	1780	992	91.03	96.01	100	425	140	680	2849
csv [13]	41	694	309	90.64	92.34	100	146	35	235	1272
exec [14]	56	407	70	54.84	78.90	100	104	27	248	327
fast-pfor [36]	82	971	82	54.55	87.40	100	127	14	748	302
fileupload [15]	49	381	39	13.02	98.31	100	121	39	192	194
graph [16]	118	879	146	58.78	97.13	100	216	29	541	975
jansi [51]	48	474	107	23.47	84.83	100	378	0	409	123
pool [17]	98	1097	73	22.29	91.60	100	203	91	682	649
validator [18]	130	1228	464	63.31	95.51	100	421	209	646	1463
Total	836	8575	2719	56.57	91.99	100	2245	641	4654	10334

these, 915 are application types (i.e., classes defined within the Java projects) and were directly resolved during skeleton construction. ALPHATrans prompts DeepSeek-Coder-33b-Instruct to resolve the remaining 882 and successfully translated 738 $((915 + 738)/1,797 = 91.99\%)$ of them: the generated types passed the syntactic and runtime check. The column *ATR* in Table 1 shows the results of automated type resolution. Because type resolution is essential to project skeleton construction, we manually checked the type mappings generated by DeepSeek-Coder-33b-Instruct and also translated the 144 unresolved types.

Through manual investigation of the automatically resolved types, we observed that DeepSeek-Coder-33b-Instruct's type resolution for 182 cases, while *correct*, can be *improved*. For example, ALPHATrans translated `java.io.File`, a class concerning file manipulation functionality to `str`. The resolved type can represent file paths in Python but lacks features for file manipulation. We suspect this translation is impacted by the Java use case provided in the prompt. While this translation is correct for the use case, we replaced it with `pathlib.Path` to have a more generic type mapping. We also augmented the type mapping with additional types in the target language for 38 types. For example, ALPHATrans translated `java.nio.Buffer` to `bytearray`, which is correct as they both provide a mutable sequence of bytes with efficient in-place modifications. However, `array.array` and `memoryview` also provide similar functionality with efficient and low-level data manipulation capabilities. Consequently, we augmented type mapping to `typing.Union[bytearray, array.array, memoryview]`. Given that type mapping can be reused, this one-time manual effort increases the chance of ALPHATrans's success on unseen projects.

Using the universal type mapping, ALPHATrans successfully creates and validates project skeletons in the target PL, achieving 100% syntax and runtime validation (column *SV* in Table 1). The skeleton validation step ensures all module imports, class structures, method signatures, and type annotations are done properly, making the subsequent steps easier. Applying ALPHATrans to unseen projects, if a class skeleton cannot be validated, ALPHATrans removes it from the target project, updates the skeleton based on the class dependencies, and proceeds to the next phase.

Summary. ALPHATrans can successfully transform projects to remove method and constructor overloading. Moreover, it can automatically translate 91.99% of the source PL types and use that to create and validate project skeletons in the target PL.

7.2.2 Effectiveness in Compositional Translation and Validation: Table 2 shows the compositional translation and validation results. The *AMF* column indicates the total number of application method fragments. The numbers in subsequent columns demonstrate the effectiveness of ALPHATrans in

Table 2. Effectiveness of ALPHATrans in repository-level code translation. Abbreviations in the table stand for **AMF**: #Application Method Fragments, **SNEF**: Source Non-Exercised Fragments, **GS**: Graal Success, **GF**: Graal Fail, **GE**: Graal Error, **TNEF**: Target Non-Exercised Fragments, **ATP**: Fragments All Test Pass, **OTF**: Fragments One Test Fail, **MTF**: Fragments Many Test Fail, **ATF**: Fragments All Test Fail, **TPR**: Test Pass Rate, **O**: Overall, **RE**: Runtime Error, **AF**: Assertion Failure, and **M1**: Number of *AMFs* that GraalVM could not execute (*GE*) but translated test fragments exercised.

Subjects	AMF	Syntax Check (%)	SNEF (%)	GraalVM			Test Translation												M1	
				GS (%)	GF (%)	GE (%)	TNEF (%)	ATP (%)	OTF (%)			MTF (%)			ATF (%)			TPR (%)	All	Some
									O	RE	AF	O	RE	AF	O	RE	AF			
cli	273	100	5.86	70.70	11.72	11.72	35.90	8.42	10.62	51.72	48.28	32.23	91.07	8.93	6.96	100	0	10.08	0	16
codec	680	98.53	8.97	38.38	32.50	20.15	65.59	4.12	4.41	60.00	40.00	12.79	55.11	44.89	4.12	75.21	24.79	9.43	11	27
csv	235	98.72	9.36	38.72	26.81	25.11	74.47	0	13.62	96.88	3.13	0	0	0	2.55	100	0	0	0	3
exec	248	100	45.16	33.47	2.02	19.35	34.27	4.44	2.02	40.00	60.00	7.26	93.36	6.64	6.85	100	0	19.29	6	9
fast-pfor	748	95.32	45.45	12.03	24.20	18.32	41.71	4.28	1.74	84.62	15.38	3.74	79.23	20.77	3.07	85.88	14.12	20.08	6	25
fileupload	192	100	86.98	8.85	1.04	3.13	1.56	3.65	6.77	30.77	69.23	1.04	91.67	8.33	0	0	0	63.44	2	3
graph	541	99.63	41.22	24.77	22.92	11.09	57.12	0	0.92	100	0	0.18	100	0	0.55	100	0	11.04	0	1
jansi	409	99.76	76.53	8.07	11.49	3.91	22.25	0.24	0.98	100	0	0	0	0	0	0	0	1.07	0	1
pool	682	100	77.71	6.01	1.32	14.96	19.35	1.61	1.03	100	0	0.29	100	0	0	0	0	6.62	4	2
validator	646	99.23	36.69	30.50	11.15	21.67	46.44	3.25	5.42	71.43	28.57	4.18	84.50	15.50	4.02	99.12	0.88	11.70	1	31
Total	4654	98.80	43.43	24.50	16.23	15.84	41.92	2.88	3.72	70.52	29.48	5.44	82.77	17.23	2.62	92.86	7.14	9.76	30	118

the translation and validation of *AMFs* only.⁴ The *Syntax Check* column indicates the percentage of *AMFs* that pass syntactic validation. Column *SNEF* shows the percentage of *AMFs* not covered by source project tests. ALPHATrans successfully generates syntactically correct code (98.80% of *AMFs* and 96.40% of all fragments—field, application, and test fragments—across ten subjects). We also observe that 43.43% of *AMFs* are not covered during the execution of any test, i.e., we cannot go beyond syntactic check and validate their runtime behavior or functional equivalence.

For 56.57% of the *AMFs* that are covered by source project tests, ALPHATrans attempts to validate their functional equivalence using GraalVM. Multi-column *GraalVM* shows the percentage of *AMFs* that GraalVM executes and successfully validates (*GS*), executes but there is a test assertion failure (*GF*), and cannot execute due to its limitation (*GE*) mentioned in §6.1. 24.50% (min=6.01% and max=70.70%), 16.23% (min=1.04% and max=32.50%), and 15.84% (min=3.13% and max=25.11%) of *AMFs* resulted in *Graal Success*, *Graal Fail*, and *Graal Error*, respectively. Note that these numbers add up to 56.57% of *AMFs* that were covered by source project tests. With respect to only covered methods, GraalVM Success is 43.30%. Furthermore, our analysis shows that a high portion of methods that are not covered by tests are either abstract methods or getter/setter methods. If their translations are syntactically correct, they are also likely functionally equivalent, which can ramp up the success rate (§7.5). Spearman Rank Order Correlation [52] indicates a strong positive correlation between method coverage (Table 1) and *GS* numbers ($\rho = 0.92$), confirming that with better method coverage, validated *AMFs* are very likely to be higher.

Regardless of GraalVM's validation for functional equivalence, ALPHATrans translates and executes the test fragments on the recomposed translated project. The *Test Translation* multi-column shows the results of test translation and execution. Column *TNEF* indicates the ratio of *AMFs* where execution of translated tests never reached them. Columns *ATP* through *ATF* show the number of *AMFs* that ALPHATrans executed using translated test fragments, categorized per the test execution results. For 2.88% of the *AMFs*, all the test fragments that covered them were marked as pass (*ATP*). For 3.72% and 5.44%, at least one test (*OTF*) or more than one test (*MTF*) failed. All the test fragments failed for 2.62% of the *AMFs*. Note that these numbers (*TNEF*, *ATP*, *OTF*, *MTF*, and *ATF*) add up to 56.57% of the *AMFs* that are covered by source project tests.

For cases with test failure, columns *RE* and *AF* show the breakdown of whether test failure was due to assertion failure or runtime error. We can observe that most test executions terminated

⁴Please refer to our artifact [33] for details of all the translation and validation of other fragments shown in Table 1.

with a runtime error due to translation bugs and never reached an assert statement. *Our manual investigation confirms that a high rate of runtime errors is due to a relatively small number of fragments with translation bugs.* Although test decomposition helps with *test translation coupling effect* (§2), there is still a high degree of runtime errors due to long call chains in these projects (the average number of methods executed per test in the original and decomposed test suites are 27.4 and 21.8, respectively). As a result, the overall pass rate, i.e., the percentage of recomposed test fragments for the translated projects that pass (column *TPR*), is low. These results also confirm the necessity of in-isolation testing through language interoperability by ALPHATrans.

We also calculated the number of AMFs that GraalVM could not execute (numbers under *GE* column) but translated test fragments exercised (column *M1*). *All* indicates the number of AMFs with all passing tests, including test fragments with assert statements, indicating the validation of functional equivalence (with respect to the source project tests). *Some* corresponds to the number of AMFs with at least one passing test, which indicates runtime validation. Overall, test translation validates the functional correctness and runtime behavior of 30 and 118 fragments that GraalVM could not exercise.

Finally, we analyzed translations using PyLint [2], which scores Python files on a scale of 0 to 10 based on how Pythonic the code is. All ALPHATrans translations achieved scores of 10, mainly because (1) LLMs inherently generate idiomatic code and (2) AlphaTrans uses Black [1] for formatting the translations extracted from the LLM response. These results confirm that the translations are all Pythonic, i.e., they adhere to Python coding standards and best practices.

Summary. ALPHATrans effectively performs compositional translation and validation of 17,874 fragments, achieving overall 96.40% syntactic correctness (98.80% for AMFs), 27.03% runtime behavior validation (GS+M1 Some), and 25.14% functional equivalence (GS+M1 All).

7.3 RQ2: Translation Bugs and Fixes

We investigated the manual effort for fixing translation bugs in a subset of studied subjects, namely *Commons-FileUpload*, *Commons-CLI*, *Commons-CSV*, and *Commons-Validator*. We also discuss some of the translation bugs and fixes for them to better illustrate the challenges in code translation.

7.3.1 Human Study. Our two human subjects were selected due to their relative familiarity with the selected projects. Their effort indicates an upper bound for the amount of time required to fix translation bugs since developers of the source projects are likely to fix the bugs better and faster. We shared with them the source program in Java, the translations by ALPHATrans, and all the reports and artifacts generated by ALPHATrans during translation.

For *Commons-FileUpload*, achieving green tests took 5.5 hours and required 120 and 114 line additions and deletions from partial translations. For *Commons-CLI*, the manual fix took 11 hours, making 614 and 1,253 line additions and deletions, respectively. The project was very dense for *Commons-CSV*, with many method calls, making it harder to manually fix bugs. Nevertheless, a developer achieved all green tests in 30 hours with 2,676 and 999 line additions and deletions, respectively. Finally, for *Commons-Validator*, the developer spent 34 hours to fix translation bugs, with 3,585 and 2,416 line additions and deletions, respectively. One of the major feedback from developers was that test decomposition greatly helped locate and fix translation bugs: in case of a test failure, developers only need to investigate the last call statement in the failed test fragment instead of looking at the stack trace and other prior calls.

7.3.2 Translation Bugs. Our artifacts [33] contain partial translations and fixed versions as separate commits. These commits can serve as useful benchmarks for evaluating fault localization, program repair, and test generation techniques. This section shows *four* instances of such translation bugs.

The two most prevalent sources of translation bugs are mismatches between APIs and behavioral differences in the PLs. The code snippet below demonstrates a bug that happened due to a mismatch in the logic of Calendar (Java) and datetime (Python). Line 3 in Java sets the MONTH field to 0, which corresponds to the first month of the year (January). Similarly, the Python translation sets the month attribute to 0; however, in the Python library, January corresponds to value 1 for month.

```
1 ----- JAVA SOURCE CODE -----
2 Calendar calendar = Calendar.getInstance()
3 calendar.set(Calendar.MONTH, 0);
```

```
1 ----- PYTHON TRANSLATION -----
2 calendar = datetime.datetime.now()
3 - calendar = calendar.replace(month=0)
4 + calendar = calendar.replace(month=1)
```

The next example shows the difference in implicit type casting between the two PLs. Line 5 in Java source code concatenates a String value with null. During execution, Java runtime silently casts null to a String and then performs the concatenation operation on it. In Python, concatenating an str with None results in a TypeError as the operands of the binary operation has different types. A correct Python translation requires explicit casting of None to str as shown in Line 5.

```
1 ----- JAVA SOURCE CODE -----
2 qChar = "...";
3 nullStr = null;
4
5 this.qNullStr = qChar + nullStr + qChar;
```

```
1 ----- PYTHON TRANSLATION -----
2 qChar = "...";
3 nullStr = None
4 - self.qNullStr = qChar + nullStr + qChar
5 + self.qNullStr = qChar + str(nullStr) + qChar
```

The third example shows an instance of write(int b) method from ByteArrayOutputStream class, where the least significant 8 bits of the integer $(b2 \ll 4) \mid (b3 \gg 2)$ are directly written to the stream. The incorrect Python translation attempts to construct a bytes object using a singleton list with the input integer before writing it to an object of type io.BytesIO. However, this neglects that the bytes() constructor requires the integers in the input iterable to be strictly in the range of [0, 255]. Thereby, a ValueError is thrown when b2 is large. The correct translation requires 0xF, a mask that maintains only the 4 lowest bits of b2 before left-shifting by 4 as shown in Line 3 under Python translation. Given that b3 and b4 each contain no more than 8 bits, this change ensures the least significant 8 bits of $(b2 \ll 4) \mid (b3 \gg 2)$ are correctly written to the BytesIO object.

```
1 ----- JAVA SOURCE CODE -----
2
3 out.write((b2 << 4) | (b3 >> 2));
```

```
1 ----- PYTHON TRANSLATION -----
2 - out.write(bytes([(b2 << 4) | (b3 >> 2)]))
3 + out.write(bytes([(b2 & 0xF) << 4 | (b3 >> 2)]))
```

The last code snippet demonstrates the Java behavior of an iterator that is unavailable in Python. The incorrect Python translation uses next() to implement both next() and hasNext() methods of the Java java.util.Iterator type. However, calling next() increments the iterator in Python. The correct translation should implement PeekableIterator interface in Python with a method hasNext() -> bool.

```
1 ----- JAVA SOURCE CODE -----
2 Iterator<String> headers = ls.keySet().iterator();
3
4 assertEquals("content", headers.next());
5
6 assertFalse(headers.hasNext());
```

```
1 ----- PYTHON TRANSLATION -----
2 - headers = iter(ls.keys())
3 + headers = PeekableIterator(ls.keys())
4 self.assertEqual("content", next(headers))
5 - self.assertFalse(next(headers, None) is not None)
6 + self.assertFalse(headers.hasNext())
```

Implications. To obtain correct translation, especially for code using library APIs, models need to generate test cases as well that can validate the translated fragment in isolation. This could be an interesting direction for applying an agentic approach, where the orchestrating agent can decide when to generate test cases, and the test case generator agent gets all the information by running static analysis tools, gathering context from previous runs, collecting API documentation by crawling the internet, and finally generating the translation based on all the information.

Summary. Although ALPHATrans cannot validate all the translations, it provides partial translations and artifacts that developers can use to complete the translation and achieve green tests in a reasonable time (20.1 hours, on average).

7.4 RQ3: Impact of Test Decomposition

We previously showed the effectiveness of test translation in validating the runtime behavior or functional correctness of translated method fragments (§7.2.2). To better understand how test decomposition helps address *test translation coupling effect*, we collected translated unit tests with at least two decomposed test fragments. We further applied filtering and kept only those tests for which *all* their decomposed fragments were executed, regardless of passing or failing outcomes. The yellow bars in Figure 6 show the percentage of selected unit tests from the translated test suites. None of the tests met the criteria for *Commons-CSV*, so we excluded it from further investigation.

We categorized the selected unit tests into two groups: those with all passing test fragments (green bars in Figure 6) and those with at least one failing test fragment (red bars in Figure 6). For the unit tests in the latter group, we calculated the pass rate of the *decomposed test fragments*. The blue box chart in Figure 6 shows the distribution of the measured pass rate per unit test. These unit tests would have been marked as *fail* without test decomposition. However, we can observe that these tests can be decomposed into test fragments such that 62.41% of them pass. These results indicate how

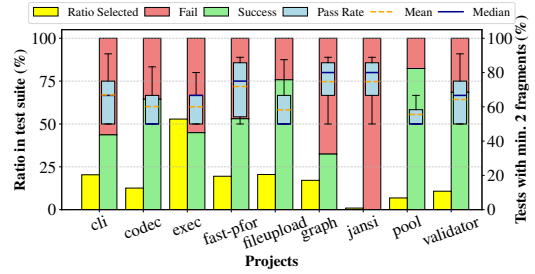


Fig. 6. Effectiveness of test decomposition in ALPHA-TRANS for validating earlier fragments in failing tests.

Summary. Test decomposition unburdens validation of translated method fragments from incorrect translations. 62.41% of test fragments for unit tests that would have been marked as failed achieved passing outcomes.

7.5 RQ4: Impact of Test Coverage

Although existing developer-written tests are useful for checking functional equivalence, they can pose two major issues for automated code translation and validation. First, the coverage for these tests can be extremely low (e.g., 13.02% for Commons-FileUpload [15] as reported in Table 1), preventing most of the code from being validated; as our investigation of RQ1 showed, the translation validation rate strongly correlates with test suites' (method) coverage. Second, a developer-written test can have a long call sequence. To show the positive impact of more-focused tests with higher coverage on translation validation, we automatically generated additional tests using EvoSuite [19]. We used default tool configuration with a time budget of 120 seconds per class.

Table 3 compares properties of the developer-written and EvoSuite-generated tests. Evosuite tests cover more methods than the developer-written test suite (66.87% *Method Coverage* compared to 56.57%). Furthermore, the average number of methods executed per single test is almost half that of decomposed test suites (11.41 compared to 21.84 methods). To demonstrate the impact of test quality on ALPHA-TRANS's overall performance, we translated and executed the EvoSuite-generated tests. Corroborated by the numbers under *TPR+* and *ATP+* columns, we can see that augmenting the test suite increases the method coverage, and thereby, the *TPR* and *ATP* numbers from RQ1 by 5.85% and 2.11%, respectively. Not all EvoSuite tests have assertions, and even if they do, the quality of the assertions could be lower compared to developer-written tests (e.g., checking trivial properties with weak fault detection ability). Nonetheless, the higher *TPR* and *ATP* of such tests enhance runtime validation, which is still promising in code translation. EvoSuite is incompatible

Table 3. Effectiveness of test augmentation in exercising and validating more application method fragments. Abbreviations in the table stand for **ATP**: Fragments All Test Pass and **TPR**: Test Pass Rate. ATP+ and TPR+ demonstrate ATP and TPR gain through test augmentation.

Subjects	Developer-Written Test					EvoSuite Test				
	Method Coverage (%)	# Decomposed Tests	Avg. Methods Executed / Test	TPR (%)	ATP (%)	Method Coverage (%)	# Tests	Avg. Methods Executed / Test	TPR+ (%)	ATP+ (%)
cli	94.14	3036	34.25	10.08	8.42	95.97	569	12.15	2.99	1.47
codec	91.03	3522	10.56	9.43	4.12	80.74	1141	8.02	3.51	0.88
csv	90.64	1219	52.62	0	0	74.04	220	39.16	0	0.00
exec	54.84	311	18.99	19.29	4.44	61.29	245	6.32	3.27	1.21
fast-pfor	54.55	249	41.62	20.08	4.28	39.17	1843	4.31	5.59	1.07
fileupload	13.02	93	3.54	63.44	3.65	70.31	231	5.29	11.26	2.60
graph	58.78	933	25.02	11.04	0.00	76.71	800	9.00	5.13	0.92
jansi	23.47	187	13.57	1.07	0.24	51.83	332	9.08	3.31	0.73
pool	22.29	287	6.52	6.62	1.61	37.24	394	7.36	10.41	2.20
validator	63.31	1479	11.68	11.70	3.25	81.42	1305	13.43	9.73	7.59
Total	56.57	11316	21.84	9.76	2.88	66.87	7080	11.41	5.85	2.11

Table 4. Importance of program transformation. Abbreviations in the table stand for **AMF**: #Application Method Fragments, **SNEF**: Source Non-Exercised Fragments, **GS**: Graal Success, **GF**: Graal Fail, **GE**: Graal Error, **TNEF**: Target Non-Exercised Fragments, **ATP**: Fragments All Test Pass, **OTF**: Fragments One Test Fail, **MTF**: Fragments Many Test Fail, **ATF**: Fragments All Test Fail, and **TPR**: Test Pass Rate.

Subjects	AMF	SNEF (%)	GraalVM			Test Translation					
			GS (%)	GF (%)	GE (%)	TNEF (%)	ATP (%)	OTF (%)	MTF (%)	ATF (%)	TPR (%)
cli	276	5.80	0	0	94.20	85.51	0	2.17	1.09	5.43	0.46
codec	678	10.62	0	0	89.38	72.86	2.06	5.46	2.80	6.19	5.04
csv	235	8.51	0	0	91.49	88.09	0.43	0.43	0.85	1.70	0.32
exec	253	46.25	26.88	4.35	22.53	49.41	1.58	1.19	0	1.58	4.29
fast-pfor	754	46.02	0	0	53.98	38.20	0.93	6.63	0.80	7.43	4.88
fileupload	168	85.12	10.71	0.60	3.57	8.93	1.19	2.98	1.19	0.60	23.08
graph	556	40.65	0	0	59.35	56.12	0	1.98	0.36	0.90	5.48
jansi	314	69.75	0	0	30.25	29.94	0	0.32	0	0	0
pool	574	70.56	0	0	29.44	27.00	1.39	0	0.35	0.70	5.48
validator	623	33.39	18.78	20.06	27.77	66.29	0	0	0.32	0	0.43
Total	4431	40.01	4.58	3.09	52.31	52.79	0.81	2.57	0.86	2.96	3.05

with Java 21, and hence GraalVM, which prevented us from using Evosuite-generated tests in ALPHATrans. We anticipate that incorporating it in ALPHATrans could improve the overall quality of translations.

Summary. Augmenting the existing test suite increases code coverage, thereby exercising and validating more AMFs. Test augmentation can further validate the correctness of 2.11% of fragments not executed by developer tests. The generated tests are more focused and, on average, invoke 48% fewer methods than the developer-written tests.

7.6 RQ5: Ablation Study

We performed three ablation studies to investigate the impact of program transformation, choice of LLM, and program decomposition on the performance of ALPHATrans.

7.6.1 Impact of Program Transformation. We removed the program transformation component of ALPHATrans and executed the entire pipeline. The results in Table 4 show that without transformation (e.g., resolving method/constructor overloading), the performance of ALPHATrans drops significantly: *GS*, *ATP*, and *TPR* values decreased to 4.58% (from 24.50%), 0.81% (from 2.88%), and 3.05% (from 9.76%), respectively. This is because Python does not support overloading and only considers the last method/constructor implementation, resulting in runtime errors or test failures. Also, *GE* values increase due to the interference of overloaded code constructs with GraalVM.

Table 5. Effectiveness of ALPHATrans with GPT-4o in compositional translation and validation. Abbreviations in the table stand for **AMF**: #Application Method Fragments, **SNEF**: Source Non-Exercised Fragments, **GS**: Graal Success, **GF**: Graal Fail, **GE**: Graal Error, **TNEF**: Target Non-Exercised Fragments, **ATP**: Fragments All Test Pass, **OTF**: Fragments One Test Fail, **MTF**: Fragments Many Test Fail, **ATF**: Fragments All Test Fail, **TPR**: Test Pass Rate, **O**: Overall, **RE**: Runtime Error, **AF**: Assertion Failure, and **M1**: Number of AMFs that GraalVM could not execute (*GE*) but translated test fragments exercised.

Subjects	Syntax Check (%)	SNEF (%)	GraalVM			Test Translation												M1		Cost (\$)
			GS (%)	GF (%)	GE (%)	TNEF (%)	ATP (%)	OTF (%)			MTF (%)			ATF (%)			TPR (%)	All	Some	
								O	RE	AF	O	RE	AF	O	RE	AF				
cli	99.63	5.86	76.92	8.79	8.42	78.39	4.76	3.66	90.00	10.00	6.23	100	0	1.10	100	0	2.47	0	1	19.69
codec	97.79	8.97	42.50	28.53	20.00	74.56	3.82	4.12	78.57	21.43	6.18	30.62	69.38	2.35	73.33	26.67	9.57	2	18	39.97
csv	98.30	9.36	41.28	25.96	23.40	86.81	0	0	0	0	1.70	78.17	21.83	2.13	96.72	3.28	0.98	0	8	16.66
exec	99.60	45.16	35.89	2.42	16.53	44.76	4.84	0	0	0	5.24	0	100	0	0	0	28.62	1	1	3.90
fast-pfor	97.46	45.45	14.71	16.98	22.86	52.94	1.47	0	0	0	0	0	0	0.13	100	0	2.41	2	0	12.93
fileupload	99.48	86.98	9.90	0.52	2.60	4.17	6.77	1.56	0	100	0.52	100	0	0	0	0	54.84	1	3	2.25
graph	98.71	41.22	27.73	19.04	12.01	58.78	0	0	0	0	0	0	0	0	0	0	0	0	0	12.03
jansi	99.76	76.53	8.07	11.98	3.42	23.47	0	0	0	0	0	0	0	0	0	0	0	0	0	3.66
pool	99.71	77.71	7.48	1.32	13.49	21.70	0.59	0	0	0	0	0	0	0	0	0	2.09	0	0	7.33
validator	99.23	36.69	38.24	15.94	9.13	54.95	4.02	2.32	93.33	6.67	1.55	70.59	29.41	0.46	100	0	10.41	0	3	25.53
Total	98.80	43.43	27.83	14.54	14.20	50.64	2.26	1.20	80.36	19.64	1.87	59.39	40.61	0.60	86.59	13.41	6.45	6	34	143.95

7.6.2 Choice of LLM. For this experiment, we replaced the DeepSeek-Coder-33b-Instruct with GPT-4o and repeated the entire pipeline of ALPHATrans (Table 5). A stronger model such as GPT-4o improves the translation quality—functional equivalence increases from 25.14% to 27.95%. For some projects, the ATP and TPR rates are higher for DeepSeek-Coder-33b-Instruct translations, whereas for the others, GPT-4o results in higher values. We investigated each LLM’s successful AMF translations to better understand the differences. We observed a huge overlap between successful AMFs and the unique benefits each LLM provides in code translation (Figure 7). GPT-4o handles API translation and type casting better, resolving the first three translation bugs discussed in §7.3. In contrast, it tends to add unnecessary code, mostly due to error handling, which results in a functional mismatch. In the example below, `create2` method can take a `None` value, and its implementation performs error handling when necessary. GPT-4o adds unnecessary error handling code, interfering with program logic and resulting in test failures.

```

1  ----- JAVA SOURCE CODE -----
2  public Option create1(OptionGroup optGroup) {
3      return create2(optGroup);
4  }
5  }

```

```

1  ----- PYTHON TRANSLATION -----
2  def create1(self, optGroup: OptionGroup) -> Option:
3      if optGroup is None:
4          raise ValueError("optGroup is None")
5      return self.create2(optGroup)

```

It is worth noting that using commercial LLMs comes at a cost. The last column of Table 5 (column *Cost*) shows the cost of using GPT-4o for repeating the experiments, resulting in the total cost of \$143.95 for translating all the subjects (average cost of \$14.39 per project).

7.6.3 Impact of Program Decomposition. For this ablation study, we prompted GPT-4o and DeepSeek-Coder-33b-Instruct *file-by-file* and evaluated translation correctness through test execution and GraalVM (Table 6). Not surprisingly, a considerable percentage of the files exceeded the model context window size, particularly for DeepSeekCoder, with 9.08% of the files encountering this problem. Among the prompted files, 21.36% (19.44% for DeepSeekCoder and 1.92% for GPT-4o) were syntactically incorrect. Translations that passed the syntactic correctness check did not pass any translated test execution, whereas GraalVM validated 2.56% of the files in total (1.02% for DeepSeekCoder and 1.54% for GPT-4o). Note that file-level GraalVM validation does not mean that all the methods are correctly translated and validated—only the methods in the class that are executed by tests. Manual analysis of the results of this experiment revealed that one prominent culprit of test failures was LLM hallucination with method/variable names. Given that we had no skeleton construction in this baseline, such issues could not be avoided; this demonstrates the usefulness of skeleton construction and incremental translation in ALPHATrans.

Table 6. Importance of program decomposition. Abbreviations stand for **GS**: Graal Success and **TPR**: Test Pass Rate. Since Java can contain multiple classes in one file, **#Files** is smaller from **#Classes** in Table 1.

Subjects	# Files	GPT-4o				DeepSeek-Coder			
		Over Context	Syntax Error	GS	TPR (%)	Over Context	Syntax Error	GS	TPR (%)
cli	51	0	0	1	0	5	6	0	0
codec	136	0	1	7	0	28	46	4	0
csv	33	0	3	0	0	6	8	0	0
exec	54	0	0	0	0	1	4	0	0
fast-pfor	85	1	1	1	0	12	30	1	0
fileupload	43	0	0	0	0	2	4	0	0
graph	159	0	6	3	0	0	17	3	0
jansi	30	0	0	0	0	3	10	0	0
pool	72	0	1	0	0	4	8	0	0
validator	119	0	3	0	0	10	19	0	0
Total	782	1	15	12	0	71	152	8	0

Summary. Omitting program transformation and program decomposition significantly lowers the effectiveness of ALPHATrans. A stronger model such as GPT-4o resolves non-trivial issues concerning type casting and API translation but may result in trivial translation bugs. When possible, users of ALPHATrans can prompt multiple LLMs to achieve better translation performance.

8 Related Work

There are generally two main categories of techniques for translating code from one PL to another: (1) using transpilers and statistical machine translation and (2) leveraging language models.

8.1 Code Translation Using Non-LLM-Based Approaches

Tools like C2Rust [26], CxGo [58], Sharpen [45], and Java2CSharp [27] translate code from C to Rust, C to Go, and Java to C# respectively. A series of statistical machine translation techniques [8, 39–41] focus on translating Java to C#. Deep learning approaches have also been applied for code translation [48, 49]. None of these efforts have tackled translating real-world Java projects to Python. LLM-based techniques are also superior to transpilers in terms of performance or readability [44].

8.2 Code Translation Using LLMs

Recently, LLMs have been employed for code translation [9, 30, 44, 57, 61, 64], demonstrating high success rates on crafted examples but poor performance on real-world projects. Other studies [4, 66] have also applied language models for code translation, mainly focusing on crafted benchmarks. Concurrently to our work, two other techniques for repository-level code translation focusing on different language pairs were proposed [50, 65]. SYZGY [50] translates repository-level C to Rust using GPT-4. Oxidizer [65] leverages language feature mapping and type-driven techniques for translating Go to Rust. Both techniques use I/O equivalence for validating their translations. There are also approaches that use transpiler output to guide LLM-based code translation [62]. However, the limitation of such work is the availability of robust and well-maintained transpilers, which, in many cases, may not be feasible. Nitin et al. [42] presented a specification-based translation, where a natural language specification is captured from the source code, which helps the translation process. Yang et al. [63] used tests to assist the translation. Compared to previous work, our contributions include a compositional and validation-guided code translation approach that leverages two types of validation techniques and evaluation of the approach on 10 real-world projects.

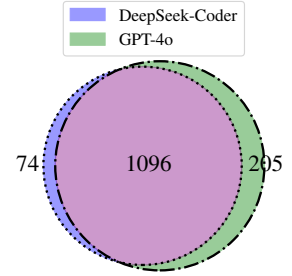


Fig. 7. Functional equivalence overlap of AMFs (Graal Success) between LLMs.

9 Threats to Validity

External Validity. One of the key external threats is the generalizability of ALPHATrans. We built and evaluated the first version of ALPHATrans to translate from Java to Python. However, our pipeline is generic, and with minimal effort, the current implementation can translate Java programs to more target languages (e.g., languages supported by GraalVM). Furthermore, the majority of the tools that we used support a large set of programming languages such as JavaScript, Ruby, C/C++, and Rust.

Internal Validity. One threat can be the manual validation of the translated types. To address that, several authors verified the types individually and consulted API documents when necessary. Another threat is that, while all successes reported by the GraalVM validation are true successes, we may have *underestimated* the capabilities of ALPHATrans by a considerable margin due to the significant proportion of errors caused by limitations in the GraalVM validation approach. To mitigate this threat, we manually augmented the universal type map to support a more comprehensive translation of types. That said, we have implemented *driver* code templates to provide a mechanism for adding the support for more types by the users of ALPHATrans if needed.

Construct Validity. In order to mitigate construct validity, ALPHATrans is built and validated with well-vetted tools, such as GraalVM [43], JaCoCo [53], Python coverage [6], CodeQL [20], etc.

10 Concluding Remarks

In this paper, we introduced ALPHATrans, a neuro-symbolic approach that combines the power of static analysis and abilities of LLMs in code synthesis to automate repository-level code translation and validation. ALPHATrans decomposes the program into smaller fragments and translates the fragments in reverse call order, incrementally building the source project in the target language. In addition to syntactic checks, ALPHATrans implements two types of validation through GraalVM and test translation. ALPHATrans is the first approach to translate and validate real-world projects, and we envision several research directions to advance repository-level code translation and validation.

One of the major challenges in repository-level code translation is identifying suitable library APIs in the target PL. Often, equivalent Python APIs may not exist, requiring new code generation or translation of the library API itself. Even if similar libraries exist, the logic of libraries might be different in two PLs. ALPHATrans supports translating frequently used APIs and aims to build a generic pipeline. Supporting all the libraries in the pipeline remains an open challenge that we aim to address in future work. Furthermore, while the idea of compositional translation and validation is PL-agnostic, the static analysis makes the extension of ALPHATrans to translating from other source projects challenging. Devising LLM-enabled or PL-agnostic static analysis approaches can benefit code translation approaches such as ALPHATrans. We also showed that the quality of the source project test suite can significantly impact the translation validation results. In future work, we plan to integrate an LLM-based test generator into the ALPHATrans pipeline to enhance the validation component.

11 Data Availability

Artifacts and implementation of ALPHATrans are publicly available [33, 34].

Acknowledgments

This work is supported by the IBM-Illinois Discovery Accelerator Institute and NSF CCF-2238045 grants. We thank Prof. Darko Marinov and Raju Pavuluri for their help with this research. We also thank the anonymous reviewers for their comments, which helped make this work stronger.

References

- [1] 2024. Black: Python Code Formatter. <https://github.com/psf/black>
- [2] 2024. PyLint Static Code Analyzer for Python. <https://pypi.org/project/pylint/>
- [3] Muhammad Salman Abid, Mrigank Pawagi, Sugam Adhikari, Xuyan Cheng, Ryed Badr, Md Wahiduzzaman, Vedant Rathi, Ronghui Qi, Choyin Li, Lu Liu, Rohit Sai Naidu, Licheng Lin, Que Liu, Asif Zubayer Palak, Mehzabin Haque, Xinyu Chen, Darko Marinov, and Saikat Dutta. 2024. GlueTest: Testing Code Translation via Language Interoperability. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 612–617. doi:10.1109/ICSME58944.2024.00061
- [4] Wasi Uddin Ahmad, Md Golam Rahman Tushar, Saikat Chakraborty, and Kai-Wei Chang. 2023. AVATAR: A Parallel Corpus for Java-Python Program Translation. In *Findings of the Association for Computational Linguistics: ACL 2023*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 2268–2281. doi:10.18653/v1/2023.findings-acl.143
- [5] Andrés Bastidas Fuertes, María Pérez, and Jaime Meza Hormaza. 2023. Transpilers: A Systematic Mapping Review of Their Usage in Research and Industry. *Applied Sciences* 13, 6 (2023). doi:10.3390/app13063667
- [6] Ned Batchelder. 2024. Coverage.py. <https://pypi.org/project/coverage>.
- [7] Dante Broggi and Yi Liu. 2023. On the Interoperability of Programming Languages via Translation. In *2023 Congress in Computer Science, Computer Engineering, & Applied Computing (CSCE)*. 2579–2585. doi:10.1109/CSCE60160.2023.00413
- [8] Xinyun Chen, Chang Liu, and Dawn Song. 2018. Tree-to-tree neural networks for program translation. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems (Montréal, Canada) (NIPS’18)*. Curran Associates Inc., Red Hook, NY, USA, 2552–2562.
- [9] Peng Di, Jianguo Li, Hang Yu, Wei Jiang, Wenting Cai, Yang Cao, Chaoyu Chen, Dajun Chen, Hongwei Chen, Liang Chen, Gang Fan, Jie Gong, Zi Gong, Wen Hu, Tingting Guo, Zhichao Lei, Ting Li, Zheng Li, Ming Liang, Cong Liao, Bingchang Liu, Jiachen Liu, Zhiwei Liu, Shaojun Lu, Min Shen, Guangpei Wang, Huan Wang, Zhi Wang, Zhaogui Xu, Jiawei Yang, Qing Ye, Gehao Zhang, Yu Zhang, Zelin Zhao, Xunjin Zheng, Hailian Zhou, Lifu Zhu, and Xianying Zhu. 2024. CodeFuse-13B: A Pretrained Multi-lingual Code Large Language Model. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice (Lisbon, Portugal) (ICSE-SEIP ’24)*. Association for Computing Machinery, New York, NY, USA, 418–429. doi:10.1145/3639477.3639719
- [10] Hadeel A. Osman Eman J. Coco and Niemah I. Osman. 2018. JPT : A Simple Java-Python Translator. *CAIJ* 5, 2 (2018), 1–18. doi:10.5121/caij.2018.5201
- [11] The Apache Software Foundation. 2024. Apache Commons CLI. <https://github.com/apache/commons-cli>
- [12] The Apache Software Foundation. 2024. Apache Commons Codec. <https://github.com/apache/commons-codec>
- [13] The Apache Software Foundation. 2024. Apache Commons CSV. <https://github.com/apache/commons-csv>
- [14] The Apache Software Foundation. 2024. Apache Commons Exec. <https://github.com/apache/commons-exec>
- [15] The Apache Software Foundation. 2024. Apache Commons FileUpload. <https://github.com/apache/commons-fileupload>
- [16] The Apache Software Foundation. 2024. Apache Commons Graph. <https://github.com/apache/commons-graph>
- [17] The Apache Software Foundation. 2024. Apache Commons Pool. <https://github.com/apache/commons-pool>
- [18] The Apache Software Foundation. 2024. Apache Commons Validator. <https://github.com/apache/commons-validator>
- [19] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (Szeged, Hungary) (ESEC/FSE ’11)*. Association for Computing Machinery, New York, NY, USA, 416–419. doi:10.1145/2025113.2025179
- [20] GitHub. 2024. CodeQL. <https://codeql.github.com>
- [21] Dony George, Priyanka Girase, Mahesh Gupta, Prachi Gupta, and Aakanksha Sharma. 2010. Programming Language Inter-conversion. *International Journal of Computer Applications* 1, 20 (February 2010), 63–69. doi:10.5120/419-619
- [22] GraalVM. 2024. Polyglot API. <https://www.graalvm.org/latest/reference-manual/polyglot-programming>.
- [23] Giovanni Guizzo, Jie M. Zhang, Federica Sarro, Christoph Treude, and Mark Harman. 2023. Mutation analysis for evaluating code translation. *Empirical Software Engineering* 29, 1 (06 Dec 2023), 19. doi:10.1007/s10664-023-10385-w
- [24] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. arXiv:2401.14196
- [25] Ali Reza Ibrahimzada. 2024. Program Decomposition and Translation with Static Analysis. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings (Lisbon, Portugal) (ICSE-Companion ’24)*. Association for Computing Machinery, New York, NY, USA, 453–455. doi:10.1145/3639478.3641226
- [26] Immunant. 2024. C2Rust Transpiler. <https://github.com/immunant/c2rust>.
- [27] Paul Irwin. 2024. Java to CSharp Converter. <https://github.com/paulirwin/JavaToCSharp>.
- [28] Suman Jain and Inderveer Chana. 2015. Modernization of Legacy Systems: A Generalised Roadmap. In *Proceedings of the Sixth International Conference on Computer and Communication Technology 2015 (Allahabad, India) (ICCCCT ’15)*.

- Association for Computing Machinery, New York, NY, USA, 62–67. doi:10.1145/2818567.2818579
- [29] Pooyan Jamshidi, Aakash Ahmad, and Claus Pahl. 2013. Cloud Migration Research: A Systematic Review. *IEEE Transactions on Cloud Computing* 1, 2 (2013), 142–157. doi:10.1109/TCC.2013.10
- [30] Mingsheng Jiao, Tingrui Yu, Xuan Li, Guanjie Qiu, Xiaodong Gu, and Beijun Shen. 2023. On the Evaluation of Neural Code Translation: Taxonomy and Benchmark. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1529–1541. doi:10.1109/ASE56229.2023.00114
- [31] Ravi Khadka, Belfrit V. Batlajery, Amir M. Saeidi, Slinger Jansen, and Jurriaan Hage. 2014. How do professionals perceive legacy systems and software modernization?. In *Proceedings of the 36th International Conference on Software Engineering (Hyderabad, India) (ICSE 2014)*. Association for Computing Machinery, New York, NY, USA, 36–47. doi:10.1145/2568225.2568318
- [32] Musawwer Khan, Islam Ali, Wasif Nisar, Muhammad Qaiser Saleem, Ali S Ahmed, Haysam E Elamin, Waqar Mehmood, and Muhammad Shafiq. 2022. Modernization Framework to Enhance the Security of Legacy Information Systems. *Intelligent Automation & Soft Computing* 32, 1 (2022), 543–555. doi:10.32604/iasc.2022.016120
- [33] Intelligent CAT Lab. 2025. AlphaTrans Artifact Website. <https://github.com/Intelligent-CAT-Lab/AlphaTrans>.
- [34] Intelligent CAT Lab. 2025. AlphaTrans Data Repository. <https://doi.org/10.5281/zenodo.15204625>.
- [35] Kevin Lano and Hanan Siala. 2024. Using model-driven engineering to automate software language translation. *Automated Software Engineering* 31, 1 (28 Feb 2024), 20. doi:10.1007/s10515-024-00419-y
- [36] Daniel Lemire. 2024. JavaFastPFOR. <https://github.com/lemire/JavaFastPFOR>
- [37] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [38] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173. doi:10.1162/tac1_a_00638
- [39] Anh Tuan Nguyen, Tung Thanh Nguyen, and Tien N Nguyen. 2013. Lexical Statistical Machine Translation for Language Migration. In *FSE*. ACM, New York, NY, USA, 651–654.
- [40] Anh Tuan Nguyen, Tung Thanh Nguyen, and Tien N. Nguyen. 2013. Lexical statistical machine translation for language migration. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (Saint Petersburg, Russia) (ESEC/FSE 2013)*. Association for Computing Machinery, New York, NY, USA, 651–654. doi:10.1145/2491411.2494584
- [41] Anh Tuan Nguyen, Tung Thanh Nguyen, and Tien N. Nguyen. 2015. Divide-and-conquer approach for multi-phase statistical migration for source code. In *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (Lincoln, Nebraska) (ASE '15)*. IEEE Press, 585–596. doi:10.1109/ASE.2015.74
- [42] Vikram Nitin, Rahul Krishna, and Baishakhi Ray. 2024. SpecTra: Enhancing the Code Translation Ability of Language Models by Generating Multi-Modal Specifications. arXiv:2405.18574
- [43] Oracle. 2024. GraalVM. <https://www.graalvm.org>.
- [44] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in Translation: A Study of Bugs Introduced by Large Language Models while Translating Code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 82, 13 pages. doi:10.1145/3597503.3639226
- [45] Mono Project. 2023. Sharpen - Automated Java->C# conversion. <https://github.com/mono/sharpen>.
- [46] pytest dev. 2024. Pytest. <https://www.pytest.org>.
- [47] Lili Qiu. 1999. *Programming Language Translation*. Technical Report. Cornell University, USA.
- [48] Baptiste Roziere, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. 2020. Unsupervised translation of programming languages. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 1730, 11 pages.
- [49] Baptiste Rozière, Jie M. Zhang, François Charton, Mark Harman, Gabriel Synnaeve, and Guillaume Lample. 2021. Leveraging Automated Unit Tests for Unsupervised Code Translation. *CoRR* abs/2110.06773 (2021). arXiv:2110.06773 <https://arxiv.org/abs/2110.06773>
- [50] Manish Shetty, Naman Jain, Adwait Godbole, Sanjit A Seshia, and Koushik Sen. 2024. Syzygy: Dual Code-Test C to (safe) Rust Translation using LLMs and Dynamic Analysis. *arXiv preprint arXiv:2412.14234* (2024).
- [51] Fuse Source. 2024. Jansi. <https://github.com/fusesource/jansi>
- [52] C. Spearman. 1904. The proof and measurement of association between two things. *The American Journal of Psychology* 15, 1 (1904), 72–101. doi:10.2307/1412159
- [53] The JaCoCo Team. 2024. Java Code Coverage. <https://www.eclemma.org/jacoco/>
- [54] The JUnit Team. 2024. JUnit. <https://junit.org/junit5/>

- [55] A.A. Terekhov and C. Verhoef. 2000. The realities of language conversions. *IEEE Software* 17, 6 (2000), 111–124. doi:10.1109/52.895180
- [56] TIOBE. 2024. TIOBE Index. <https://www.tiobe.com/tiobe-index>.
- [57] Sindhu Tipirneni, Ming Zhu, and Chandan K. Reddy. 2024. StructCoder: Structure-Aware Transformer for Code Generation. *ACM Trans. Knowl. Discov. Data* 18, 3, Article 70 (Jan. 2024), 20 pages. doi:10.1145/3636430
- [58] Go Transpile. 2024. C to Go Translator. <https://github.com/gotranspile/cxgo>.
- [59] Tree-Sitter. 2024. Tree-Sitter Library. <https://tree-sitter.github.io/tree-sitter/>
- [60] Thomas Würthinger, Christian Wimmer, Andreas Wöß, Lukas Stadler, Gilles Duboscq, Christian Humer, Gregor Richards, Doug Simon, and Mario Wolczko. 2013. One VM to rule them all. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software* (Indianapolis, Indiana, USA) (*Onward! 2013*). Association for Computing Machinery, New York, NY, USA, 187–204. doi:10.1145/2509578.2509581
- [61] Weixiang Yan, Yuchen Tian, Yunzhe Li, Qian Chen, and Wen Wang. 2023. CodeTransOcean: A Comprehensive Multilingual Benchmark for Code Translation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 5067–5089. doi:10.18653/v1/2023.findings-emnlp.337
- [62] Aidan ZH Yang, Yoshiki Takashima, Brandon Paulsen, Josiah Dodds, and Daniel Kroening. 2024. VERT: Verified Equivalent Rust Transpilation with Large Language Models as Few-Shot Learners. arXiv:2404.18852
- [63] Zhen Yang, Fang Liu, Zhongxing Yu, Jacky Wai Keung, Jia Li, Shuo Liu, Yifan Hong, Xiaoxue Ma, Zhi Jin, and Ge Li. 2024. Exploring and Unleashing the Power of Large Language Models in Automated Code Translation. *Proc. ACM Softw. Eng.* 1, FSE, Article 71 (July 2024), 24 pages. doi:10.1145/3660778
- [64] Xin Yin, Chao Ni, Tien N Nguyen, Shaohua Wang, and Xiaohu Yang. 2024. Rectifier: Code Translation with Corrector via LLMs. arXiv:2407.07472
- [65] Hanliang Zhang, Cristina David, Meng Wang, Brandon Paulsen, and Daniel Kroening. 2024. Scalable, validated code translation of entire projects using large language models. *arXiv preprint arXiv:2412.08035* (2024).
- [66] Ming Zhu, Karthik Suresh, and Chandan K Reddy. 2022. Multilingual Code Snippets Training for Program Translation. *Proceedings of the AAAI Conference on Artificial Intelligence* 36, 10 (Jun. 2022), 11783–11790. doi:10.1609/aaai.v36i10.21434

Received 2024-09-13; accepted 2025-04-01